

Delegate all your tests to an AI

Valentin Dumas



Tests

Important

Vital

“

**I devote at least 25 % of
dev time into testing**

”

“

**Could I be 25 %
more productive ?**

”



Valentin Dumas

Fullstack Developer, *Augmented*



Focus of the day

Delegate all your tests to an AI

Focus of the day

~~end-to-end testing, load testing,
UI testing~~

UT

IT



Off-the-shelf solutions

Delegate all your tests to an AI

Off-the-shelf solutions



ChatGPT

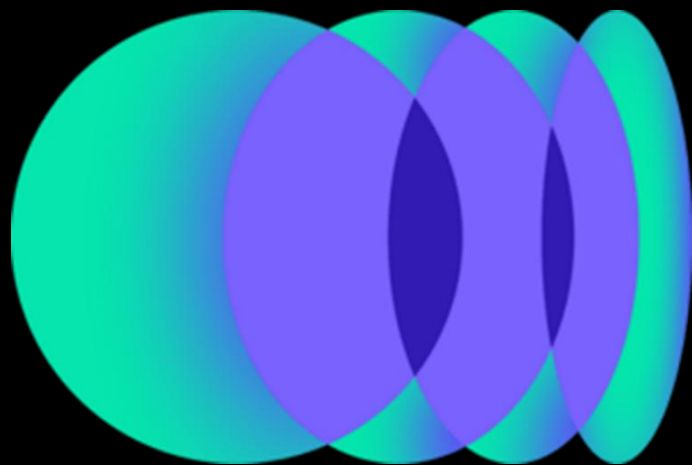
Gemini



GitHub Copilot



Cody AI



codium^{ai}

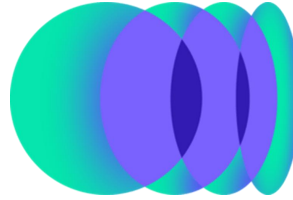
Delegate all your tests to an AI

CodiumAI : How does it work ?



“Generate tests”

“Refacto this class”



codium^{ai}



**PROJECT
CONTEXT**



GPT 3.5/4





Live Demo

IT generation on Controller

“Generate integration tests for endpoints using rest template.

Write whole class ”

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
```

Codiumate: Add more tests

```
public class BankAccountControllerIT {  
    ... @LocalServerPort  
    private int port;  
    ... @Autowired  
    private TestRestTemplate restTemplate;  
    ... @Autowired  
    private JdbcTemplate jdbcTemplate;  
    private String baseUrl;  
  
    ... @BeforeEach  
    public void setUp() {  
        ... this.baseUrl = "http://localhost:" + port + "/accounts";  
        ... jdbcTemplate.execute( sql: "TRUNCATE TABLE accounts");  
    }  
}
```

IT generation on controller

“Generate integration tests for endpoints using rest template.

Write whole class ”

```

1  @Test
2  Codiumate: Add more tests
3  public void testGetAccountNotFound() {
4      ResponseEntity<BankAccount> response = restTemplate.getForEntity(
5          url: baseUrl + "/nonexistent", BankAccount.class);
6      assertThat(response.getStatusCode()).isEqualTo(HttpStatus.NOT_FOUND);
7  }
8
9  @Test
10 Codiumate: Add more tests
11 public void testWithdrawNotFound() {
12     String accountNumber = "nonexistent";
13     double amount = 50.0;
14     ResponseEntity<BankAccount> response = restTemplate.postForEntity(
15         url: baseUrl + "/" + accountNumber + "/withdraw?amount=" + amount,
16         assertThat(response.getStatusCode()).isEqualTo(HttpStatus.NOT_FOUND);
17 }

```




Outcome

Outcome

what you could get out of it

→ TU generation

~~1-2h~~

→ TI generation on Service

1 - 30 min

→ TI generation on Controller

15 - 20 %

GLOBAL

time

“



”

Delegate all your tests to an AI

What's great

- Developer Experience
- TU use case correctly addressed
- **Functional** with libraries (Spring, MockMVC, URLs)
- AI complies with **conventions** and knows how to **mock**
- Correctly **Factorises**

Delegate all your tests to an AI

System limitations

For the developer experience

→ it would be worth moving towards **autocompletion** (IntelliJ)

→ no **self-hosting** (yet)

Delegate all your tests to an AI

System limitations

Technical aspects

- By default, AI does **little to factorize** its generated code.
(unless specifically instructed to “Factor into constants”)
- Long writing of prompts for IT

Delegate all your tests to an AI

Codium AI limitations



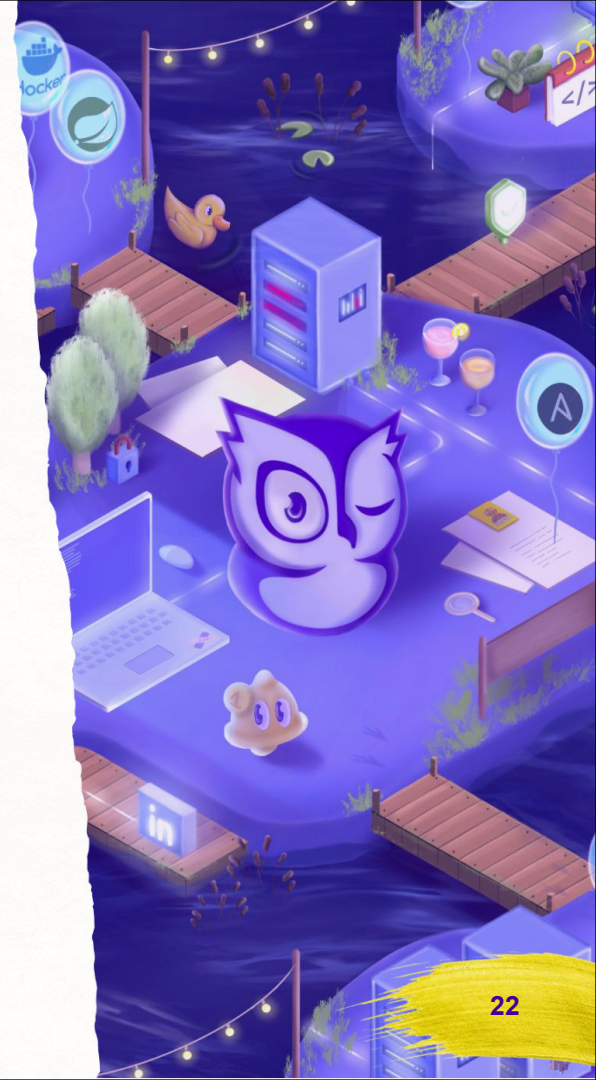
Technical



Intellectual Property



Confidentiality



Delegate all your tests to an AI

How I use it on a daily basis

It's my best **boilerplate** generator

- For a new project / service
- For a new test class

It's a **real assistant** for UT / IT

- For complex refactoring / use cases
- Code suggestions

“

Yes

”

“

We can **save** at least
15% of our time !

”

“

**Generative AI can already
be integrated
to **save time on testing****

”

“

However..

”

“

**These approaches do not replace
the need to fully understand
the “architecture”
(at least for now)**

”

“

Architecture is for me,

Boilerplate for AI

”

Delegate all your tests to an AI

Thank you for your attention



Valentin Dumas

Fullstack Developer, *Augmented*



